

Sistem Düşüncesi: Bir Görselleştirme

Ömer Faruk Yavuz

Haziran 2026

Giriş

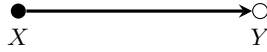
Bu metin, sistem düşüncesinin çekirdek fikirlerini — doğrusal ve sistemsel düşünme ayrımı, geri besleme, gecikme ve kaldıraç hiyerarşisi — görsel bir dille sunar. Kavramsal arka plan için “Sistem Teorisi: Arketipler, Kaldıraç Noktaları ve Karmaşıklık” belgesine bakılabilir.

I — İki Düşünme Biçimi

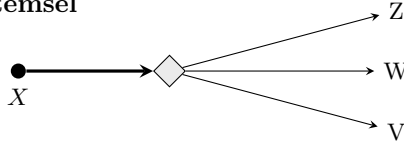
Doğrusal düşünce: bir eylem, bir sonuç. Dolu nokta aktif eylem, boş nokta sonuç. “Düğmeye bastım, ışık yandı.” Mühendis zihninin default ayarı, çoğu post-mortem’in örtük modeli.

Sistemsel düşünce: bir eylem, sistemin tepkisi ($\diamond$), birden fazla dallanan sonuç. Z, W, V — aynı eylemin paralel, farklı düzlemlerdeki etkileri.

doğrusal



sistemsel



V. bölümle farkı: V *zinciri* gösterir ($X \rightarrow Z \rightarrow W$, zaman içinde ardışık), I *dallanmayı* gösterir ($X \rightarrow \{Z, W, V\}$, aynı anda farklı yerlerde). Gerçek sistemlerde ikisi de olur — hem dallanır, hem her dal kendi içinde zincir başlatır.

Somut örnek: “X’a auth ekledim” (X).

- $Z \rightarrow$ ESPN kullanıcıları tek tıkla giriyor (beklenen sonuç)

- $W \rightarrow$ dış vendor'lar artık giremiyor, ayrı bir auth flow gerekiyor (beklenmedik)
- $V \rightarrow$ session timeout davranışı değişti, uzun streaming işleri kırılıyor (gecikmeli yan etki)

Doğrusal düşünme biçimi yalnızca beklenen sonucu (Z) gözlemler ve görevi tamamlanmış kabul eder. Sistemsel düşünme biçimi ise beklenen sonucu gözlemlediğinde, eylemin diğer düzlemlerde doğurduğu sonuçları (W ve V) araştırmaya yönelir.

II — Her Şey Bir Sistem

Sistem düşüncesinin temel kabulü, birbirinden tümüyle farklı görünen alanların ortak yapısal yasalara tabi olmasıdır. Beden, aile, şirket, trafik, ekonomi ve yazılım — hepsi etkileşen parçalardan oluşur, geri besleme döngüleri barındırır, kendi dengesini korumaya çalışır ve müdahalelere benzer biçimlerde tepki verir.

Bu yüzden bir alanda öğrenilen sistem davranışı, çoğu zaman bir başka alana taşınabilir: bedendeki homeostaz ile bir yazılım sisteminin kendini stabilize etme eğilimi, trafikteki sıkışıklık ile dağıtık bir sistemdeki tıkanma, aile içindeki geri besleme ile bir organizasyondaki teşvik döngüleri aynı yapısal mantığı paylaşır. Sistem teorisinin gücü de buradan gelir: incelenen nesne değişse bile, davranışı yöneten ilkeler büyük ölçüde sabit kalır.

beden	trafik
aile	ekonomi
şirket	yazılım

— aynı yasalar geçerli —

III — Geri Besleme: Duş Örneği

Geri besleme döngüsü, bir sistemin çıktısının tekrar girdisine dönerek sonraki davranışı etkilemesidir. Duş, bunun en sezgisel örneğidir, çünkü döngünün her halkasını günlük hayatta doğrudan deneyimleriz.

Döngü şöyle işler: musluğu çevirirsin, su ısısı değişir, bu su bir süre sonra cildine ulaşır, hissin değişir ve bu hisse göre musluğu yeniden çevirirsin. Çıktı (cildindeki sıcaklık hissi), girdiye (musluğu çevirme kararına) geri besleniyor — kapalı bir döngü.

Bu döngüyü ilginç — ve sinir bozucu — kılan şey **gecikmedir**. Musluğu çevirmen ile suyun gerçekten cildine ulaşması arasında bir süre geçer. Bu gecikme yüzünden müdahalenin sonucunu anında göremezsin.

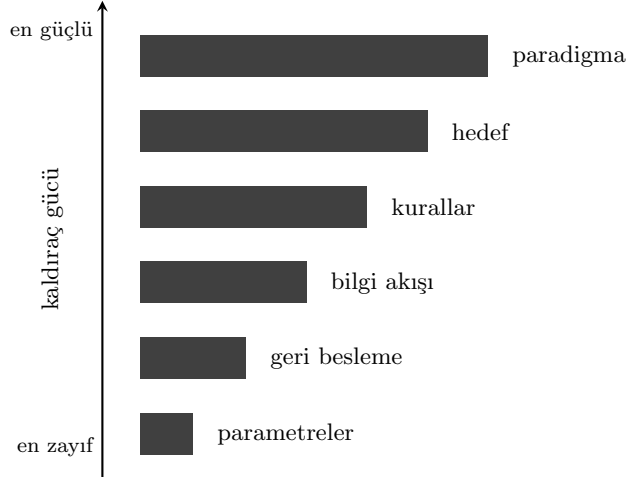
Sorun da burada başlar: sonucu hemen hissetmediğin için “yeterince çevirmedim” diye düşünüp daha fazla çevirirsin. Oysa ilk müdahalenin etkisi henüz yoldadır. Gecikmeli etki sonunda geldiğinde, üst üste binmiş iki müdahalenin toplamıyla karşılaşırısın — su birden kavurucu olur. Geri çekersin, aynı gecikme bu kez ters yönde işler ve bu sefer üşürsün. Sistem, hedeflediğin sıcaklığın etrafında bir türlü oturamaz; sıcak ile soğuk arasında salınma girer.

Buradaki genel ilke şudur: **gecikmeli bir geri besleme döngüsünde aşırı tepki, sistemi dengeye değil salınma götürür**. Sabırsızlandıkça — yani gecikmeyi hesaba katmadan müdahaleyi büyüttükçe — salınının genliği artar. Çözüm çoğu zaman daha sert değil, daha küçük müdahaleler yapıp gecikmenin etkisini görmeyi beklemektir.

Aynı kalıp duşla sınırlı değildir: bir ekonomide faiz kararlarının etkisi aylar sonra hissedilir, bir yazılım sisteminde ölçeklendirme tepkisi metriklere gecikmeli yansır, bir ekipte yapılan bir değişikliğin sonucu haftalar sonra ortaya çıkar. Gecikmeyi göz ardı eden her müdahale, aynı salınım riskini taşır.

IV — Kaldıraç Hiyerarşisi

Donella Meadows’un meşhur “kaldıraç noktaları” çerçevesinin sadeleştirilmiş hâli. Bir sistemi değiştirmek istediğinde nereye dokunabileceğinin hiyerarşisi — ve her seviyenin etki gücü çok farklı.



Çubukların uzunluğu, o seviyeye yapılan müdahalenin sistem üzerindeki etki gücünü temsil eder. En altta, etkisi en sınırlı olan *parametreler* (sayılar, limitler, eşikler) yer alır; yukarı çıktıkça sırasıyla *geri besleme* döngüleri, *bilgi akışı*, *kurallar*, sistemin *hedefi* ve en tepede onu doğuran *paradigma* gelir. Aşağıdan yukarıya doğru her seviyeye müdahale etmek giderek zorlaşır, ancak başarılıldığında etkisi kat kat büyür. Pratikteki çelişki şudur: insanlar zamanlarının büyük kısmını en alttaki zayıf kaldıraçlarda harcar, oysa sistemin kaderini belirleyen kararlar üst seviyelerde alınır.

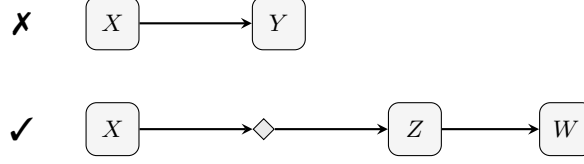
Yukarıdan aşağıya ne demek:

- **Paradigma** — “biz neyiz, ne yapıyoruz?” sorusunun cevabı. Sistemin kendine bakışı.
- **Hedef** — sistem neyi maksimize ediyor? “hızlı kullanılabilirlik” mi “güvenli arşivleme” mi optimize ediyor? Hedefi değiştirmek tüm tasarımı yeniden çizer.
- **Kurallar** — teşvikler, kısıtlar, izinler. Kim ne yapabilir, ne yapamaz. Role-based access bu seviyede.
- **Bilgi akışı** — kim neyi ne zaman görür? Standup notlarını LaTeX’e dökme alışkanlığın tam bu seviyede çalışır.
- **Geri besleme** — döngülerin gücü, tampon büyüklüğü, gecikmeler. Retry sayısı, cache TTL’i, alarm eşikleri.

- **Parametreler** — düz sayılar. Pixel, timeout, limit. En aşağı, en zayıf seviye.

Çubukların genişliği o seviyenin kaldıraç gücüdür. Yukarı çıktıkça az insan müdahale eder, ama her müdahalenin etkisi büyür. Meadows’un asıl çarpıcı tespiti: insanların neredeyse tamamı en alttaki iki seviyede yaşar — sayı ayarlar, tampon büyütür. “Bu butonu 2px büyüt” 6. seviye; “bu sistem aslında neyi optimize ediyor?” 1–2. seviye. Aynı saatler, on kat farklı etki.

V — Esas Ders



Bu, tüm görselleştirmenin özüdür: “X yaptım, sistem tepki verdi, sonra başka bir şey oldu” fikrinin sembolik hâli.

Doğrusal düşünce (✗). “X yaptım, Y oldu.” Tek bir adım, tek bir sonuç; eylemle sonucu doğrudan ve birebir eşleştirir, sonra meseleyi kapanmış sayar.

Sistemsal düşünce (✓). “X yaptım, araya bir sistem girdi (), sistem buna kendince tepki verdi, Z ortaya çıktı, Z de W’yi tetikledi.” Burada eylem ile gerçek sonuç arasında bir aracı — sistemin kendisi — vardır; etkiler tek bir noktada durmaz, zincirleme yayılır.

İki düşünme biçiminin farkı, somut bir örnekte netleşir. Diyelim ki bir bug’ı kökünden düzeltmek yerine etrafından dolaşan bir çözüm uyguladın (X). Doğrusal beklenti basittir: özellik çabucak çıkar (Y), iş biter. Sistemsal gerçek ise zincirleme işler: ekipteki herkes zamanla aynı “etrafından dolaşma” alışkanlığını benimser (Z), bu alışkanlık teknik borcu sessizce biriktirir (W) ve aylar sonra, en basit değişiklik bile neredeyse imkânsız hâle gelir. Asıl sonuç, ilk eylemden hiç beklemediğin bir noktada ve hiç beklemediğin bir zamanda ortaya çıkar.

Esas ders budur: bir eylemin sonucunu tek bir çıktıyla eşleştirmek yerine, o eylemin içinden geçtiği sistemi ve sistemin doğuracağı gecikmeli, dolaylı etkileri hesaba katmak. “X yaptım, Y oldu” düşüncesinden, “X yaptım, sistem şöyle tepki verdi, sonra şu oldu” düşüncesine geçmek.